



**QUESTÃO 1) (3,0 PONTOS)** Desenvolva uma função em C que receba um vetor de inteiros e retorne o número que mais aparece no vetor e quantas vezes ele aparece. Se houver dois elementos que aparecem o mesmo número de vezes, a função deverá retornar o maior deles. A função implementada deverá obrigatoriamente seguir o protótipo que é definido abaixo.

```
void mais_vezes(int *vet, int n, int *valor, int *vezes)
```

Faça também o programa principal que chama essa função e escreva o número que mais aparece no vetor e quantas vezes ele aparece. Considere a existência de uma função para leitura do vetor, que possui o seguinte protótipo:

```
void le_vetor(int *vet, int n)
```

**QUESTÃO 2) (3,5 PONTOS)** Desenvolver um programa C que leia uma string que contém uma data no formato "dd/mm/aaaa hh:mm:ss". O programa deverá chamar uma função que receba como parâmetro a string lida e o endereço de duas estruturas, definidas abaixo:

```
struct horario{
    int h;
    int m;
    int s;
};
typedef struct horario HORARIO;

struct data{
    int d;
    int m;
    int a;
};
typedef struct data DATA;
```

A função implementada deverá obrigatoriamente seguir o protótipo que é definido abaixo.

```
void converte_string(char *string, HORARIO *horario, DATA *data)
```

Após a chamada da função, o programa principal deverá escrever todos os campos das estruturas do tipo HORARIO e DATA. Por exemplo, para a string "01/01/2023 12:30:45", o programa deverá escrever:

```
Horário: 12:30:45
Data: 01/01/2023
```



**QUESTÃO 3) (3,5 PONTOS)** Considere um vetor de estruturas do tipo TABELA que contém as informações relativas aos N times (o valor de N é definido através da diretiva #define) que disputam um campeonato de futebol. Os campos da estrutura são definidos conforme descrito a seguir:

```
struct tabela {
    char nome[50];
    int vitorias;
    int empates;
    int derrotas;
};
typedef struct tabela TABELA;
```

Faça uma função em C que receba o vetor do tipo TABELA, um valor *n* com o tamanho do vetor e número inteiro T. A função deverá retornar um vetor com os T primeiros times classificados, em ordem de classificação. As equipes são classificadas de acordo com o número de pontos obtidos, ou seja, a equipe com mais pontos fica melhor classificado. Na pontuação, a vitória vale 3 pontos, o empate vale 1 ponto e a derrota vale 0.. Em caso de empate no número de pontos, o critério de desempate é o número de vitórias. A equipe com mais vitórias fica em posição mais alta na classificação.

A função implementada deverá obrigatoriamente seguir o protótipo que é definido abaixo.

```
TABELA * calcula_classificados(TABELA *v, int n, int T)
```

Faça também o programa principal que chama essa função e escreva os nomes dos T primeiros times classificados. Considere a existência de uma função para leitura do vetor do tipo TABELA, que possui o seguinte protótipo:

```
void le_times(TABELA *v, int n)
```



Questão 1:

```
VOID MAIS_VEZES (int *vet, int N, int *VALOR, int *vezes) {  
    int i, j, cont=0, cont_aux=0, v;  
    for (i=0; i<N; i++) {  
        for (j=0; j<N; j++) {  
            if (vet[i] == vet[j]) { cont_aux++; }  
        }  
        if (cont_aux > cont) {  
            cont = cont_aux;  
            v = vet[i];  
        } else if (cont_aux == cont && vet[i] > v) { v = vet[i]; }  
        cont_aux = 0;  
    }  
    *VALOR = v;  
    *vezes = cont;  
}
```

```
int main() {  
    int N, VALOR, vezes;  
    scanf("%d", &N); // num de elem. do vetor  
    int *vet = (int *) malloc (N * sizeof(int));  
    if (vet == NULL) {  
        printf("ERRO DE ALOCAÇÃO.");  
        exit(0);  
    }  
}
```



CONT. Q1:

le\_vetor(vet, N);

MAIS\_VEZES(vet, N, &VALOR, &VEZES);

PRINTF("Num: %d \n Vezes: %d", VALOR, VEZES);

Free(vet);

RETURN 0;



```

Questão 2: void converte_string(char *string, HORARIO *horario, DATA *data) {
    (*data).D = ((string[0] - '0') * 10) + (string[1] - '0');
    (*data).M = ((string[3] - '0') * 10) + (string[4] - '0');
    (*data).A = ((string[6] - '0') * 1000) + ((string[7] - '0') * 100) +
                ((string[8] - '0') * 10) + (string[9] - '0');

    (*horario).H = ((string[11] - '0') * 10) + (string[12] - '0');
    (*horario).M = ((string[14] - '0') * 10) + (string[15] - '0');
    (*horario).S = ((string[17] - '0') * 10) + (string[18] - '0');
}

int main() {
    char str[20];
    gets(str);

    HORARIO horario;
    DATA data;

    converte_string(str, &horario, &data);

    printf("Horario: %.d: %.d: %.d \n", horario.H, horario.M, horario.S);
    printf("Data: %.d / %.d / %.d", data.D, data.M, data.A);

    return 0;
}

```



```

Questão 3: TABELA * CALCULA_CLASSIFICADOS (TABELA * v, int N, int t){
    int SOMA_PONTOS_A = 0, SOMA_PONTOS_B = 0, i, j;
    TABELA AUX;
    FOR (i = 0; i < N - 1; i++) {
        FOR (j = 0; j < N - i - 1; j++) {
            SOMA_PONTOS_A += v[j].vitorias * 3;
            SOMA_PONTOS_A += v[j].empates;
            SOMA_PONTOS_B += v[j+1].vitorias * 3;
            SOMA_PONTOS_B += v[j+1].empates;

            IF (SOMA_PONTOS_B > SOMA_PONTOS_A) {
                AUX = v[j];
                v[j] = v[j+1];
                v[j+1] = AUX;
            } ELSE IF (SOMA_PONTOS_A == SOMA_PONTOS_B && v[j+1].vitorias >
                v[j].vitorias) {
                AUX = v[j];
                v[j] = v[j+1];
                v[j+1] = AUX;
            }

            SOMA_PONTOS_A = 0;
            SOMA_PONTOS_B = 0;
        }
    }

    TABELA * CLASSIFICADOS = (TABELA *) malloc(t * sizeof(TABELA));
    IF (CLASSIFICADOS == NULL) {
        PRINTF("ERRO DE ALOCAÇÃO CLASSIFICADOS.");
        EXIT(0);
    }
}

```



```
FOR (i=0; i<t; i++) {
```

```
    CLASSIFICADOS[i] = v[i];
```

```
}
```

```
RETURN CLASSIFICADOS;
```

```
}
```

```
int main() {
```

```
    int i, t;
```

```
    TABELA v[N];
```

```
    le_times(v, N);
```

```
    scanf("%d", &t); // quant. times CLASSIFICADOS
```

```
    TABELA * CLASSIFICADOS = calcula_classificados(v, N, t);
```

```
    printf("\n %d times CLASSIFICADOS:\n", t);
```

```
    FOR (i=0; i<t; i++) {
```

```
        printf("%d\n", CLASSIFICADOS[i].nome);
```

```
    }
```

```
    free(CLASSIFICADOS);
```

```
    return 0;
```

```
}
```